Q5 – Functions, Parameters and Scope

Mutable default bug

A default list is created once when the function is defined. Calls to add_item_buggy share that same list, so a later customer sees earlier items. Use None as the default and create a new list inside the function, or require the caller to pass an order explicitly.

Parameters and returns

Positional arguments are matched by position; keyword arguments name a parameter; defaults make optional parameters possible. A function can return multiple values as a tuple, such as subtotal, GST and total.

Scope

Local names exist inside a function. global allows rebinding a module-level name; nonlocal allows rebinding a name in an enclosing function. Mutating a referenced list does not require global, but rebinding a global name does. Prefer returning updated values rather than changing global state.

Complexity

Appending one item is amortised O(1); storing m items for one customer uses O(m) space. A closure can hold customer-specific state but should be created separately per customer.

Python Program

```
def add_item_buggy(item, order=[]):
    order.append(item)
    return order

def add_item(item, order=None):
    if order is None:
        order = []
    order.append(item)
    return order

def bill(items, gst_rate=0.05):
    subtotal = sum(items)
    gst = subtotal * gst_rate
    return subtotal, gst, subtotal + gst

if __name__ == "__main__":
    print(add_item_buggy("Tea"), add_item_buggy("Vada"))
    print(add_item("Tea"), add_item("Vada"))
    print(bill([30, 20]))
    print(add_item_buggy.__defaults__)
```